

Lecture-2

perceptron learning

Learning Rules

- Supervised Learning

Network is provided with a set of examples of proper network behavior (inputs/targets)

$$\{\mathbf{p}_1, \mathbf{t}_1\}, \{\mathbf{p}_2, \mathbf{t}_2\}, \dots, \{\mathbf{p}_Q, \mathbf{t}_Q\}$$

- Reinforcement Learning

Network is only provided with a grade, or score, which indicates network performance

- Unsupervised Learning

Only network inputs are available to the learning algorithm. Network learns to categorize (cluster) the inputs.

Reinforcement learning is similar to supervised learning, except that, instead of being provided with the correct output for each network input, the algorithm is only given a grade. The grade (or score) is a measure of the network performance over some sequence of inputs. This type of learning is currently much less common than supervised learning. It appears to be

In *unsupervised learning*, the weights and biases are modified in response to network inputs only. There are no target outputs available. At first

Prototype Vectors

$$\mathbf{p} = \begin{bmatrix} \textit{shape} \\ \textit{texture} \\ \textit{weight} \end{bmatrix}.$$

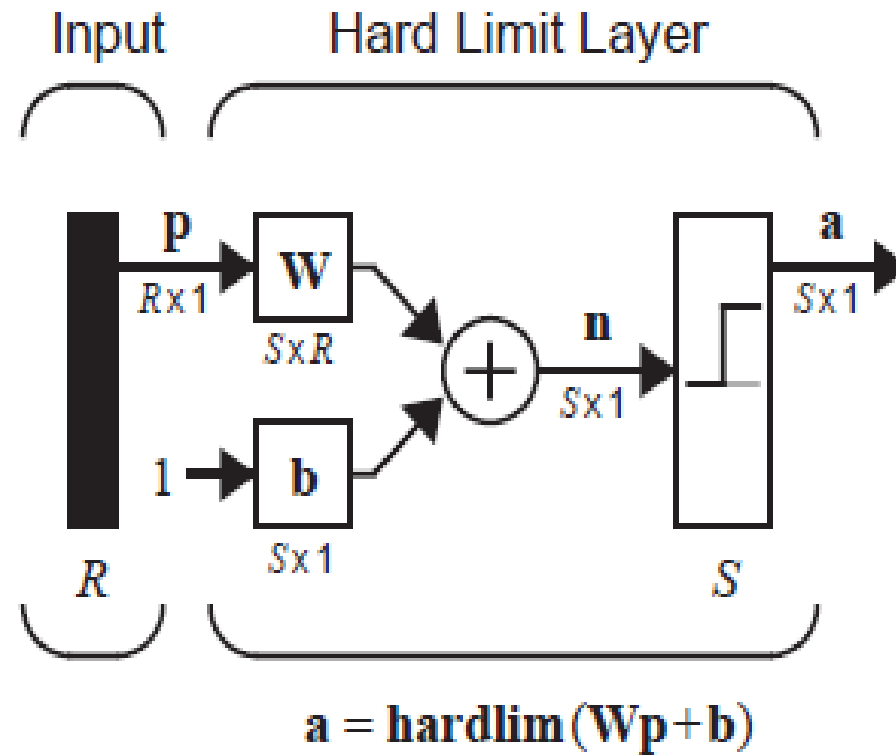
Therefore, a prototype orange would be represented by

$$\mathbf{p}_1 = \begin{bmatrix} 1 \\ -1 \\ -1 \end{bmatrix},$$

and a prototype apple would be represented by

$$\mathbf{p}_2 = \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix}.$$

Perceptron Architecture



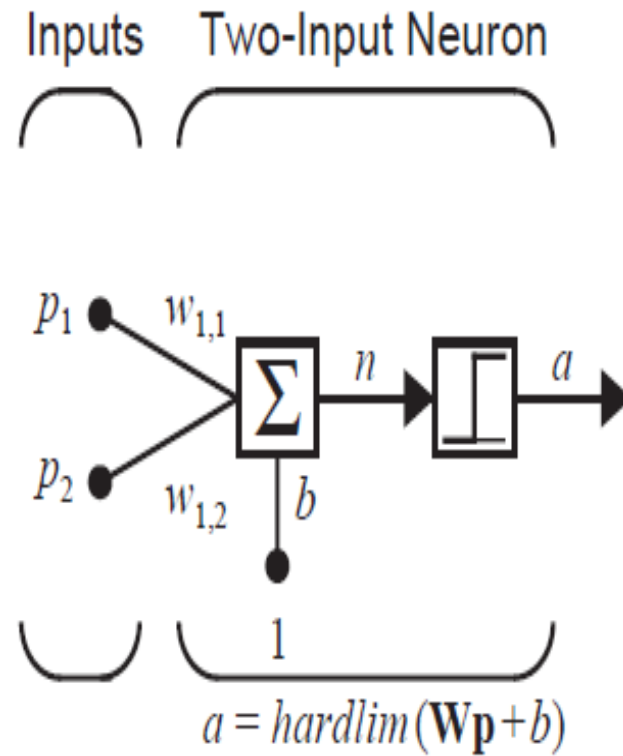
$$\mathbf{W} = \begin{bmatrix} w_{1,1} & w_{1,2} & \cdots & w_{1,R} \\ w_{2,1} & w_{2,2} & \cdots & w_{2,R} \\ \vdots & \vdots & & \vdots \\ w_{S,1} & w_{S,2} & \cdots & w_{S,R} \end{bmatrix}$$

$${}_i\mathbf{w} = \begin{bmatrix} w_{i,1} \\ w_{i,2} \\ \vdots \\ w_{i,R} \end{bmatrix}$$

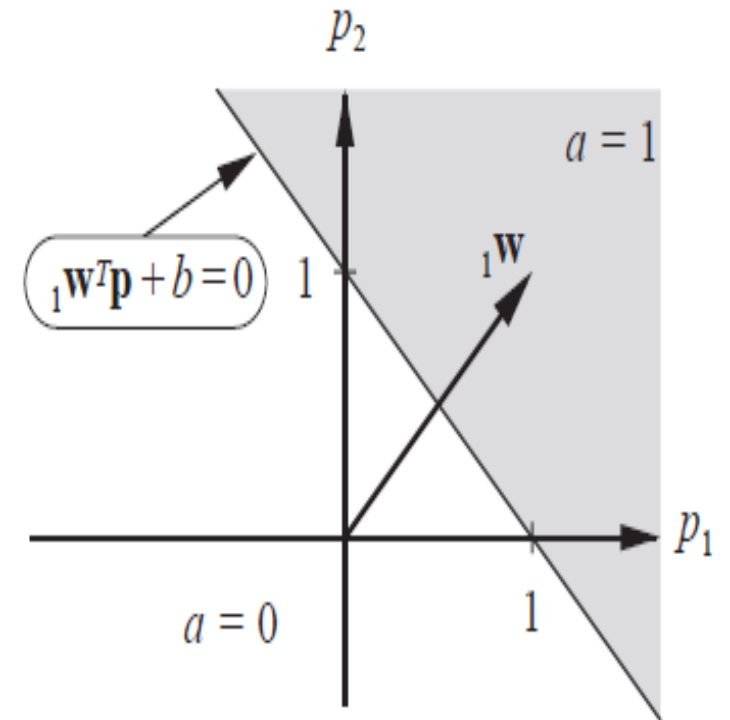
$$\mathbf{W} = \begin{bmatrix} {}_1\mathbf{w}^T \\ {}_2\mathbf{w}^T \\ \vdots \\ {}_S\mathbf{w}^T \end{bmatrix}$$

$$a_i = \text{hardlim}(n_i) = \text{hardlim}({}_i\mathbf{w}^T \mathbf{p} + b_i)$$

Single-Neuron Perceptron



$$w_{1,1} = 1 \quad w_{1,2} = 1 \quad b = -1$$



$$a = \text{hardlim}({}_1\mathbf{W}^T \mathbf{p} + b) = \text{hardlim}(w_{1,1}p_1 + w_{1,2}p_2 + b)$$

$$w_{1,1} = 1, w_{1,2} = 1, b = -1. \quad (4.10)$$

The decision boundary is then

$$n = {}_1\mathbf{w}^T \mathbf{p} + b = w_{1,1}p_1 + w_{1,2}p_2 + b = p_1 + p_2 - 1 = 0. \quad (4.11)$$

This defines a line in the input space. On one side of the line the network output will be 0; on the line and on the other side of the line the output will be 1. To draw the line, we can find the points where it intersects the p_1 and p_2 axes. To find the p_2 intercept set $p_1 = 0$:

$$p_2 = -\frac{b}{w_{1,2}} = -\frac{-1}{1} = 1 \quad \text{if } p_1 = 0. \quad (4.12)$$

To find the p_1 intercept, set $p_2 = 0$:

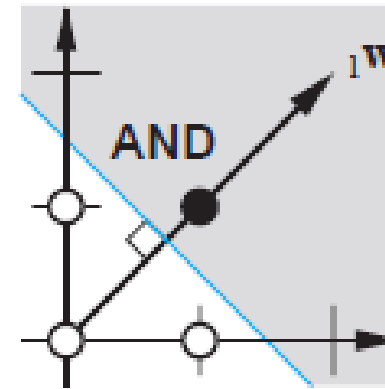
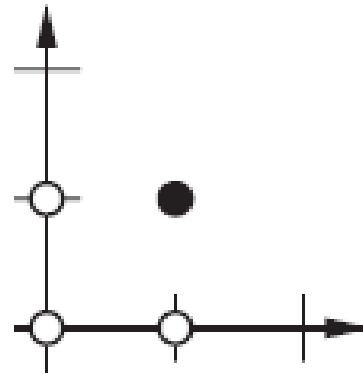
$$p_1 = -\frac{b}{w_{1,1}} = -\frac{-1}{1} = 1 \quad \text{if } p_2 = 0. \quad (4.13)$$

To find out which side of the boundary corresponds to an output of 1, we just need to test one point. For the input $\mathbf{p} = \begin{bmatrix} 2 & 0 \end{bmatrix}^T$, the network output will be

$$a = \text{hardlim}({}_1\mathbf{w}^T \mathbf{p} + b) = \text{hardlim}\left(\begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 0 \end{bmatrix} - 1\right) = 1. \quad (4.14)$$

Example AND gate for perceptron and graphical representation

$$\left\{ \mathbf{p}_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, t_1 = 0 \right\} \left\{ \mathbf{p}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, t_2 = 0 \right\} \left\{ \mathbf{p}_3 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, t_3 = 0 \right\} \left\{ \mathbf{p}_4 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, t_4 = 1 \right\}.$$



Next we want to choose a weight vector that is orthogonal to the decision boundary. The weight vector can be any length, so there are infinite possibilities. One choice is

$${}_1\mathbf{w} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}, \quad (4.16)$$

Multiple- Neuron Perceptron

Each neuron will have its own decision boundary.

$${}_i\mathbf{w}^T \mathbf{p} + b_i = 0$$

A single neuron can classify input vectors
into two categories.

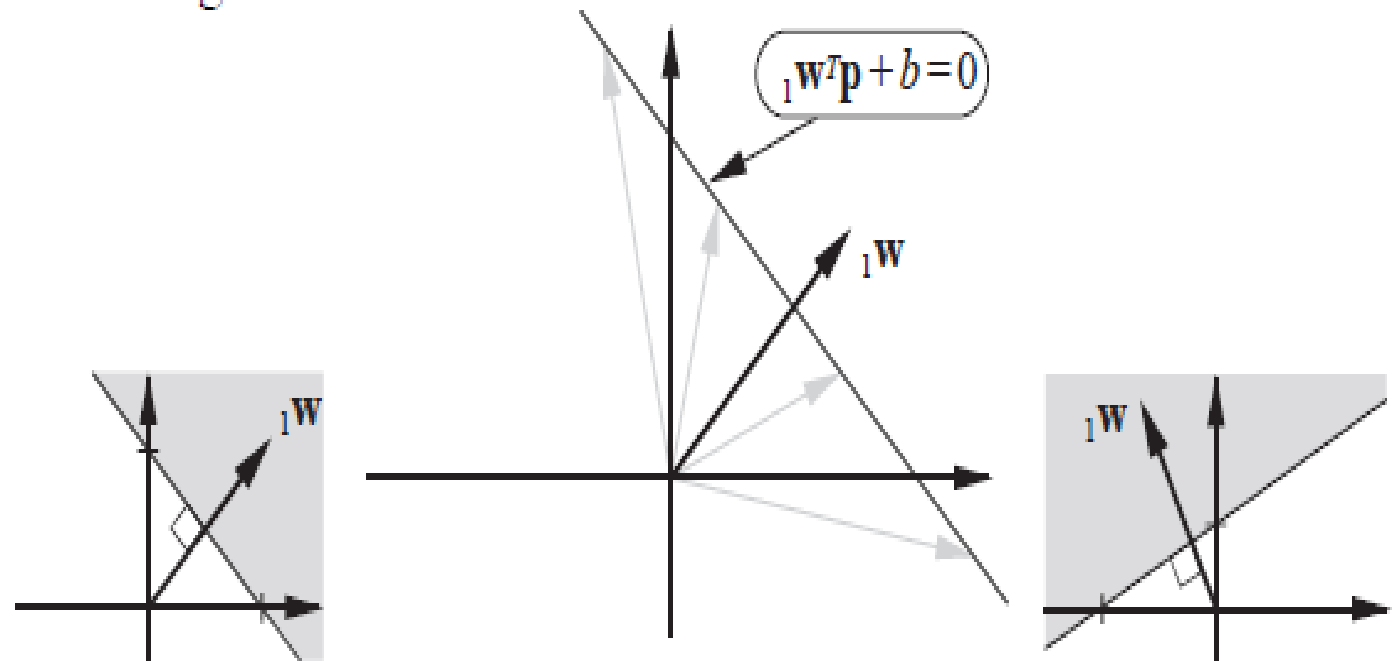
A multi-neuron perceptron can classify
input vectors into 2^S categories.

Decision Boundary

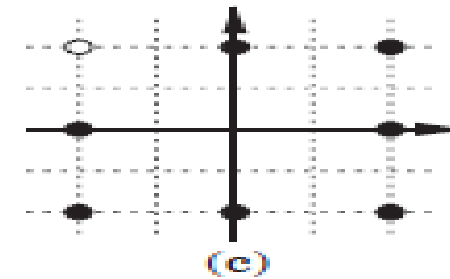
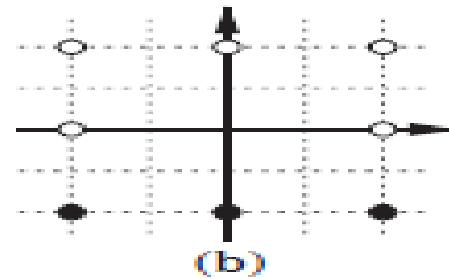
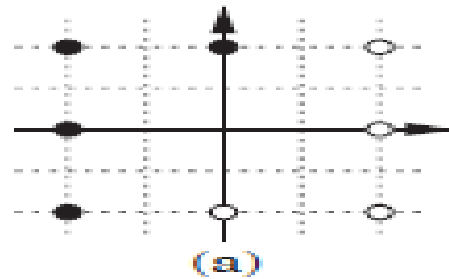
$${}_1\mathbf{w}^T \mathbf{p} + b = 0$$

$${}_1\mathbf{w}^T \mathbf{p} = -b$$

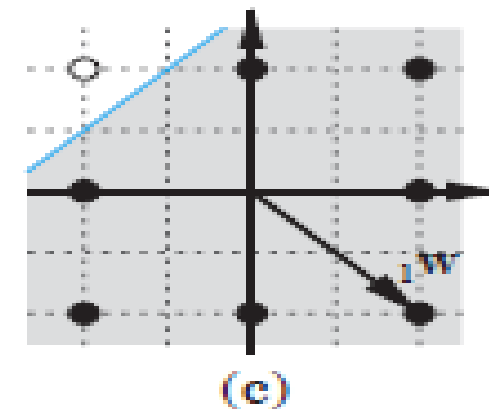
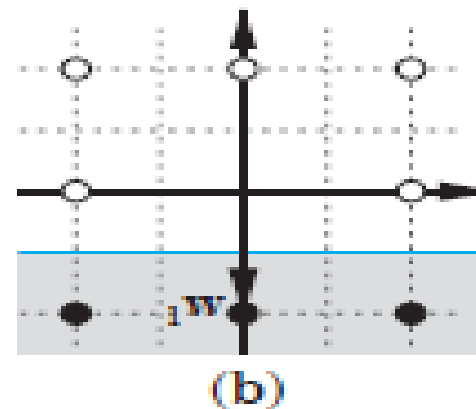
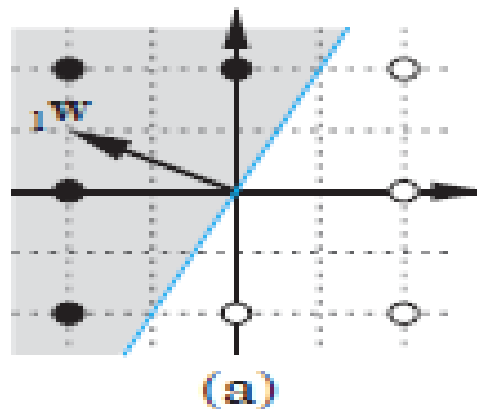
- All points on the decision boundary have the same inner product with the weight vector.
- Therefore they have the same projection onto the weight vector, and they must lie on a line orthogonal to the weight vector



P4.1 Solve the three simple classification problems shown in Figure P4.1 by drawing a decision boundary. Find weight and bias values that result in single-neuron perceptrons with the chosen decision boundaries.



The next step is to find the weights and biases. The weight vectors must be orthogonal to the decision boundaries, and pointing in the direction of points to be classified as 1 (the dark points). The weight vectors can have any length we like.



Here is one set of choices for the weight vectors:

$$\text{(a) } {}_1\mathbf{w}^T = \begin{bmatrix} -2 & 1 \end{bmatrix}, \quad \text{(b) } {}_1\mathbf{w}^T = \begin{bmatrix} 0 & -2 \end{bmatrix}, \quad \text{(c) } {}_1\mathbf{w}^T = \begin{bmatrix} 2 & -2 \end{bmatrix}.$$

$${}_1\mathbf{w}^T \mathbf{p} + b = 0$$

$$b = -{}_1\mathbf{w}^T \mathbf{p}$$

This gives us the following three biases:

$$\text{(a) } b = -\begin{bmatrix} -2 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} = 0, \text{ (b) } b = -\begin{bmatrix} 0 & -2 \end{bmatrix} \begin{bmatrix} 0 \\ -1 \end{bmatrix} = -2, \text{ (c) } b = -\begin{bmatrix} 2 & -2 \end{bmatrix} \begin{bmatrix} -2 \\ 1 \end{bmatrix} = 6$$

We can now check our solution against the original points. Here we test the first network on the input vector $\mathbf{p} = \begin{bmatrix} -2 & 2 \end{bmatrix}^T$.

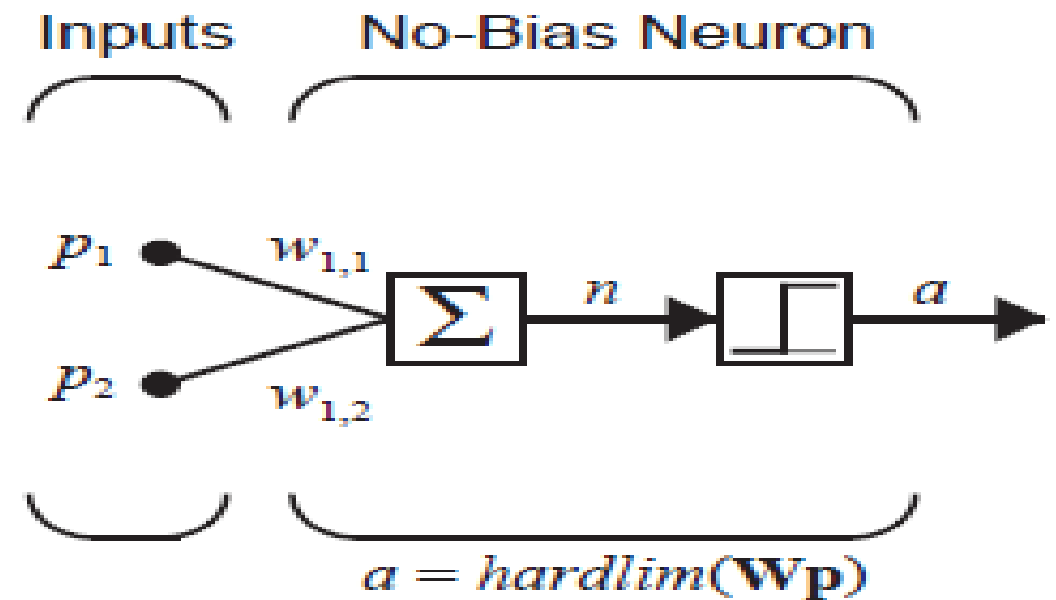
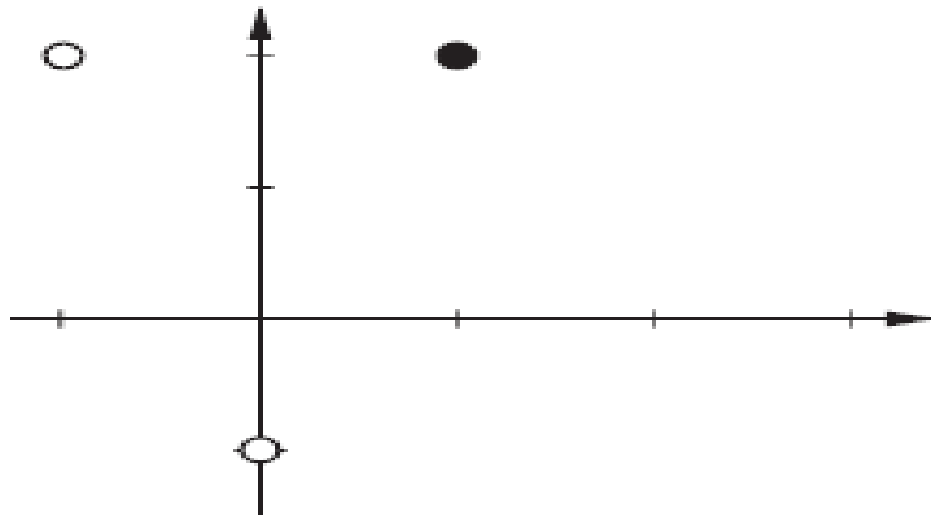
$$a = \text{hardlim}({}_1\mathbf{w}^T \mathbf{p} + b)$$

$$= \text{hardlim}\left(\begin{bmatrix} -2 & 1 \end{bmatrix} \begin{bmatrix} -2 \\ 2 \end{bmatrix} + 0\right)$$

Learning Rule Test Problem

$$\{\mathbf{p}_1, \mathbf{t}_1\}, \{\mathbf{p}_2, \mathbf{t}_2\}, \dots, \{\mathbf{p}_Q, \mathbf{t}_Q\}$$

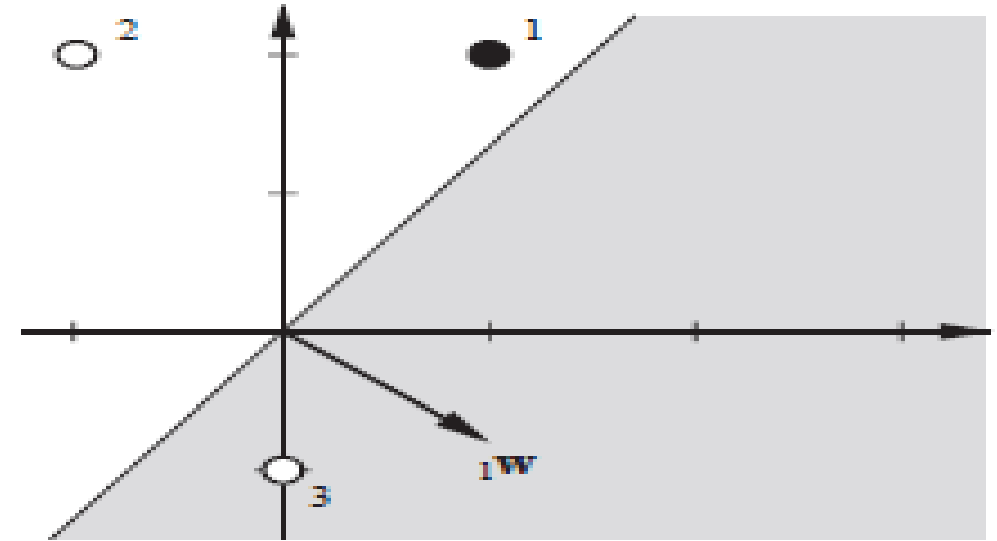
$$\left\{ \mathbf{p}_1 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, t_1 = 1 \right\} \quad \left\{ \mathbf{p}_2 = \begin{bmatrix} -1 \\ 2 \end{bmatrix}, t_2 = 0 \right\} \quad \left\{ \mathbf{p}_3 = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, t_3 = 0 \right\}$$



Starting point

Random initial weight:

$${}_1\mathbf{w} = \begin{bmatrix} 1.0 \\ -0.8 \end{bmatrix}$$



Present $\mathbf{p}_1$ to the network:

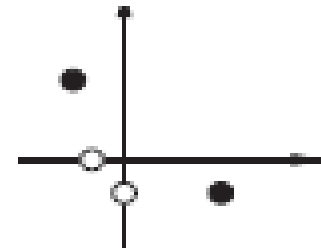
$$a = \text{hardlim}({}_1\mathbf{w}^T \mathbf{p}_1) = \text{hardlim}\left(\begin{bmatrix} 1.0 & -0.8 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix}\right)$$

$$a = \text{hardlim}(-0.6) = 0$$

Incorrect Classification.

- Set ${}_1\mathbf{w}$ to $\mathbf{p}_1$
– Not stable

×

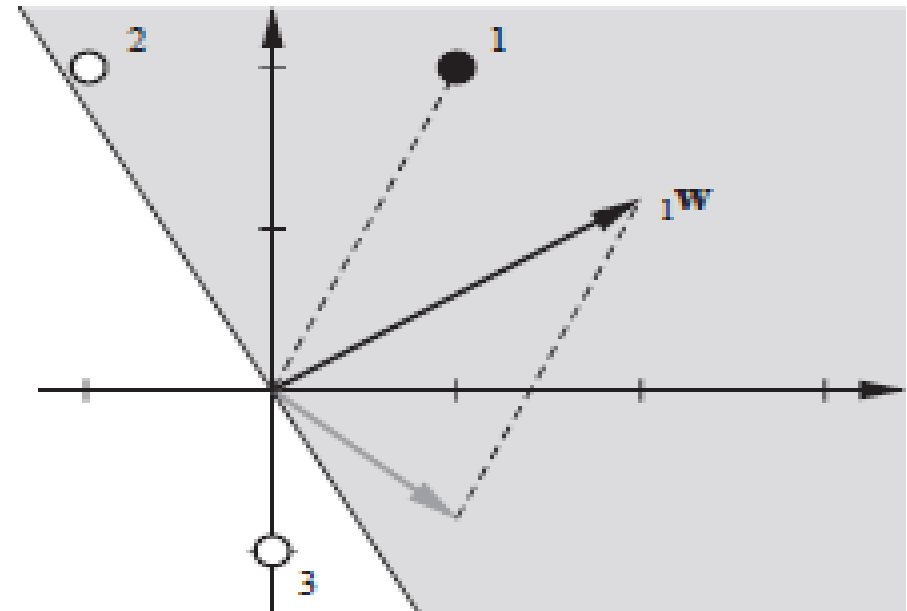


- Add $\mathbf{p}_1$ to ${}_1\mathbf{w}$

✓

Tentative Rule: If $t = 1$ and $\alpha = 0$, then ${}_1\mathbf{w}^{new} = {}_1\mathbf{w}^{old} + \mathbf{p}$

$${}_1\mathbf{w}^{new} = {}_1\mathbf{w}^{old} + \mathbf{p}_1 = \begin{bmatrix} 1.0 \\ -0.8 \end{bmatrix} + \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 2.0 \\ 1.2 \end{bmatrix}$$



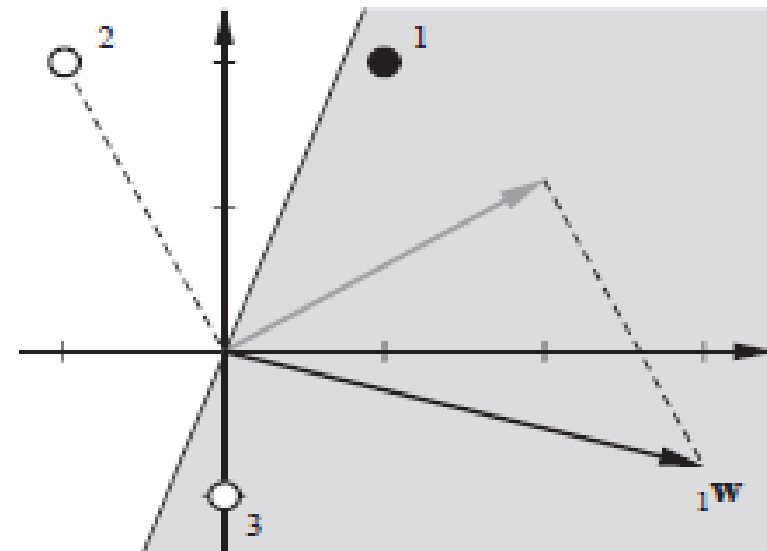
Second input vector

$$a = \text{hardlim}({}_1\mathbf{w}^T \mathbf{p}_2) = \text{hardlim}\left(\begin{bmatrix} 2.0 & 1.2 \end{bmatrix} \begin{bmatrix} -1 \\ 2 \end{bmatrix}\right)$$

$$a = \text{hardlim}(0.4) = 1 \quad (\text{Incorrect Classification})$$

Modification to Rule: If $t = 0$ and $a = 1$, then ${}_1\mathbf{w}^{new} = {}_1\mathbf{w}^{old} - \mathbf{p}$

$${}_1\mathbf{w}^{new} = {}_1\mathbf{w}^{old} - \mathbf{p}_2 = \begin{bmatrix} 2.0 \\ 1.2 \end{bmatrix} - \begin{bmatrix} -1 \\ 2 \end{bmatrix} = \begin{bmatrix} 3.0 \\ -0.8 \end{bmatrix}$$

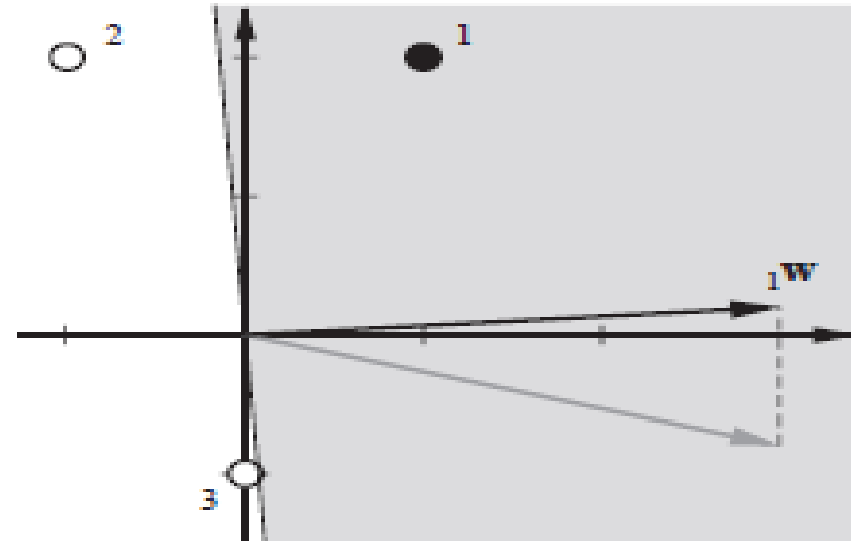


Third input vector

$$a = \text{hardlim}({}_1\mathbf{w}^T \mathbf{p}_3) = \text{hardlim}\left(\begin{bmatrix} 3.0 & -0.8 \end{bmatrix} \begin{bmatrix} 0 \\ -1 \end{bmatrix}\right)$$

$$a = \text{hardlim}(0.8) = 1 \quad (\text{Incorrect Classification})$$

$${}_1\mathbf{w}^{new} = {}_1\mathbf{w}^{old} - \mathbf{p}_3 = \begin{bmatrix} 3.0 \\ -0.8 \end{bmatrix} - \begin{bmatrix} 0 \\ -1 \end{bmatrix} = \begin{bmatrix} 3.0 \\ 0.2 \end{bmatrix}$$



Patterns are now correctly classified.

$$\text{If } t = a, \text{ then } {}_1\mathbf{w}^{new} = {}_1\mathbf{w}^{old}.$$

Unified Learning Rule

If $t = 1$ and $a = 0$, then $\mathbf{w}^{new} = \mathbf{w}^{old} + \mathbf{p}$

If $t = 0$ and $a = 1$, then $\mathbf{w}^{new} = \mathbf{w}^{old} - \mathbf{p}$

If $t = a$, then $\mathbf{w}^{new} = \mathbf{w}^{old}$

$$e = t - a$$

If $e = 1$, then $\mathbf{w}^{new} = \mathbf{w}^{old} + \mathbf{p}$

If $e = -1$, then $\mathbf{w}^{new} = \mathbf{w}^{old} - \mathbf{p}$

If $e = 0$, then $\mathbf{w}^{new} = \mathbf{w}^{old}$

$$\mathbf{w}^{new} = \mathbf{w}^{old} + e\mathbf{p} = \mathbf{w}^{old} + (t - a)\mathbf{p}$$

$$b^{new} = b^{old} + e$$

A bias is a weight with an input of 1.

Multiple-Neuron Perceptron's

To update the i th row of the weight matrix:

$${}_i\mathbf{w}^{new} = {}_i\mathbf{w}^{old} + e_i\mathbf{p}$$

$$b_i^{new} = b_i^{old} + e_i$$

Matrix form:

$$\mathbf{W}^{new} = \mathbf{W}^{old} + \mathbf{e}\mathbf{p}^T$$

$$\mathbf{b}^{new} = \mathbf{b}^{old} + \mathbf{e}$$

Apple/Banana example

Training Set

$$\left\{ \mathbf{p}_1 = \begin{bmatrix} -1 \\ 1 \\ -1 \end{bmatrix}, t_1 = \boxed{1} \right\} \quad \left\{ \mathbf{p}_2 = \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix}, t_2 = \boxed{0} \right\}$$

Initial Weights

$$\mathbf{W} = [0.5 \ -1 \ -0.5] \quad b = 0.5$$

First Iteration

$$a = \text{hardlim}(\mathbf{W}\mathbf{p}_1 + b) = \text{hardlim}\left([0.5 \ -1 \ -0.5] \begin{bmatrix} -1 \\ 1 \\ -1 \end{bmatrix} + 0.5\right)$$

$$a = \text{hardlim}(-0.5) = 0 \quad e = t_1 - a = 1 - 0 = 1$$

$$\mathbf{W}^{new} = \mathbf{W}^{old} + e\mathbf{p}^T = [0.5 \ -1 \ -0.5] + (1)[-1 \ 1 \ -1] = [-0.5 \ 0 \ -1.5]$$

$$b^{new} = b^{old} + e = 0.5 + (1) = 1.5$$

Second iteration

$$a = \text{hardlim}(\mathbf{W}\mathbf{p}_2 + b) = \text{hardlim}\left(\begin{bmatrix} -0.5 & 0 & -1.5 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix} + (1.5)\right)$$

$$a = \text{hardlim}(2.5) = 1$$

$$e = t_2 - a = 0 - 1 = -1$$

$$\mathbf{W}^{new} = \mathbf{W}^{old} + e\mathbf{p}^T = \begin{bmatrix} -0.5 & 0 & -1.5 \end{bmatrix} + (-1)\begin{bmatrix} 1 & 1 & -1 \end{bmatrix} = \begin{bmatrix} -1.5 & -1 & -0.5 \end{bmatrix}$$

$$b^{new} = b^{old} + e = 1.5 + (-1) = 0.5$$

check

$$a = \text{hardlim}(\mathbf{W}\mathbf{p}_1 + b) = \text{hardlim}\left(\begin{bmatrix} -1.5 & -1 & -0.5 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \\ -1 \end{bmatrix} + 0.5\right)$$

$$a = \text{hardlim}(1.5) = 1 = t_1$$

$$a = \text{hardlim}(\mathbf{W}\mathbf{p}_2 + b) = \text{hardlim}\left(\begin{bmatrix} -1.5 & -1 & -0.5 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix} + 0.5\right)$$

$$a = \text{hardlim}(-1.5) = 0 = t_2$$

P4.4 Solve the following classification problem with the perceptron rule. Apply each input vector in order, for as many repetitions as it takes to ensure that the problem is solved. Draw a graph of the problem only after you have found a solution.

$$\left\{ \mathbf{p}_1 = \begin{bmatrix} 2 \\ 2 \end{bmatrix}, t_1 = 0 \right\} \left\{ \mathbf{p}_2 = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, t_2 = 1 \right\} \left\{ \mathbf{p}_3 = \begin{bmatrix} -2 \\ 2 \end{bmatrix}, t_3 = 0 \right\} \left\{ \mathbf{p}_4 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}, t_4 = 1 \right\}$$

Use the initial weights and bias:

$$\mathbf{W}(0) = \begin{bmatrix} 0 & 0 \end{bmatrix} \quad b(0) = 0.$$

We start by calculating the perceptron's output a for the first input vector $\mathbf{p}_1$, using the initial weights and bias.

$$\begin{aligned} a &= \text{hardlim}(\mathbf{W}(0)\mathbf{p}_1 + b(0)) \\ &= \text{hardlim}\left(\begin{bmatrix} 0 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \end{bmatrix} + 0\right) = \text{hardlim}(0) = 1 \end{aligned}$$

The output a does not equal the target value t_1 , so we use the perceptron rule to find new weights and biases based on the error.

$$e = t_1 - a = 0 - 1 = -1$$

$$\mathbf{W}(1) = \mathbf{W}(0) + e\mathbf{p}_1^T = \begin{bmatrix} 0 & 0 \end{bmatrix} + (-1) \begin{bmatrix} 2 & 2 \end{bmatrix} = \begin{bmatrix} -2 & -2 \end{bmatrix}$$

$$b(1) = b(0) + e = 0 + (-1) = -1$$

We now apply the second input vector $\mathbf{p}_2$, using the updated weights and bias.

$$a = \text{hardlim}(\mathbf{W}(1)\mathbf{p}_2 + b(1))$$

$$= \text{hardlim}\left(\begin{bmatrix} -2 & -2 \end{bmatrix} \begin{bmatrix} 1 \\ -2 \end{bmatrix} - 1\right) = \text{hardlim}(1) = 1$$

This time the output a is equal to the target t_2 . Application of the perceptron rule will not result in any changes.

$$\mathbf{W}(2) = \mathbf{W}(1)$$

$$b(2) = b(1)$$

We now apply the third input vector.

$$\begin{aligned}
 a &= \text{hardlim}(\mathbf{W}(2)\mathbf{p}_3 + b(2)) \\
 &= \text{hardlim}\left(\begin{bmatrix} -2 & -2 \end{bmatrix} \begin{bmatrix} -2 \\ 2 \end{bmatrix} - 1\right) = \text{hardlim}(-1) = 0
 \end{aligned}$$

The output in response to input vector $\mathbf{p}_3$ is equal to the target t_3 , so there will be no changes.

$$\begin{aligned}
 \mathbf{W}(3) &= \mathbf{W}(2) \\
 b(3) &= b(2)
 \end{aligned}$$

We now move on to the last input vector $\mathbf{p}_4$.

$$\begin{aligned}
 a &= \text{hardlim}(\mathbf{W}(3)\mathbf{p}_4 + b(3)) \\
 &= \text{hardlim}\left(\begin{bmatrix} -2 & -2 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \end{bmatrix} - 1\right) = \text{hardlim}(-1) = 0
 \end{aligned}$$

This time the output a does not equal the appropriate target t_4 . The perceptron rule will result in a new set of values for $\mathbf{W}$ and b .

$$\begin{aligned}
 e &= t_4 - a = 1 - 0 = 1 \\
 \mathbf{W}(4) &= \mathbf{W}(3) + e\mathbf{p}_4^T = \begin{bmatrix} -2 & -2 \end{bmatrix} + (1) \begin{bmatrix} -1 & 1 \end{bmatrix} = \begin{bmatrix} -3 & -1 \end{bmatrix} \\
 b(4) &= b(3) + e = -1 + 1 = 0
 \end{aligned}$$

We now must check the first vector $\mathbf{p}_1$ again. This time the output a is equal to the associated target t_1 .

$$\begin{aligned}
 a &= \text{hardlim}(\mathbf{W}(4)\mathbf{p}_1 + b(4)) \\
 &= \text{hardlim}\left(\begin{bmatrix} -3 & -1 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \end{bmatrix} + 0\right) = \text{hardlim}(-8) = 0
 \end{aligned}$$

Therefore there are no changes.

$$\begin{aligned}
 \mathbf{W}(5) &= \mathbf{W}(4) \\
 b(5) &= b(4)
 \end{aligned}$$

The second presentation of $\mathbf{p}_2$ results in an error and therefore a new set of weight and bias values.

$$\begin{aligned} a &= \text{hardlim}(\mathbf{W}(5)\mathbf{p}_2 + b(5)) \\ &= \text{hardlim}\left(\begin{bmatrix} -3 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ -2 \end{bmatrix} + 0\right) = \text{hardlim}(-1) = 0 \end{aligned}$$

Here are those new values:

$$\begin{aligned} e &= t_2 - a = 1 - 0 = 1 \\ \mathbf{W}(6) &= \mathbf{W}(5) + e\mathbf{p}_2^T = \begin{bmatrix} -3 & -1 \end{bmatrix} + (1) \begin{bmatrix} 1 & -2 \end{bmatrix} = \begin{bmatrix} -2 & -3 \end{bmatrix} \\ b(6) &= b(5) + e = 0 + 1 = 1. \end{aligned}$$

Cycling through each input vector once more results in no errors.

$$\begin{aligned} a &= \text{hardlim}(\mathbf{W}(6)\mathbf{p}_3 + b(6)) = \text{hardlim}\left(\begin{bmatrix} -2 & -3 \end{bmatrix} \begin{bmatrix} -2 \\ 2 \end{bmatrix} + 1\right) = 0 = t_3 \\ a &= \text{hardlim}(\mathbf{W}(6)\mathbf{p}_4 + b(6)) = \text{hardlim}\left(\begin{bmatrix} -2 & -3 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \end{bmatrix} + 1\right) = 1 = t_4 \\ a &= \text{hardlim}(\mathbf{W}(6)\mathbf{p}_1 + b(6)) = \text{hardlim}\left(\begin{bmatrix} -2 & -3 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \end{bmatrix} + 1\right) = 0 = t_1 \\ a &= \text{hardlim}(\mathbf{W}(6)\mathbf{p}_2 + b(6)) = \text{hardlim}\left(\begin{bmatrix} -2 & -3 \end{bmatrix} \begin{bmatrix} 1 \\ -2 \end{bmatrix} + 1\right) = 1 = t_2 \end{aligned}$$

Therefore the algorithm has converged. The final solution is:

$$\mathbf{W} = \begin{bmatrix} -2 & -3 \end{bmatrix} \quad b = 1.$$

Now we can graph the training data and the decision boundary of the solution. The decision boundary is given by

$$n = \mathbf{W}\mathbf{p} + b = w_{1,1}p_1 + w_{1,2}p_2 + b = -2p_1 - 3p_2 + 1 = 0.$$

To find the p_2 intercept of the decision boundary, set $p_1 = 0$:

$$p_2 = -\frac{b}{w_{1,2}} = -\frac{1}{-3} = \frac{1}{3} \quad \text{if } p_1 = 0.$$

Assignment

E4.8 We want to train a perceptron network with the following training set:

$$\left\{ \mathbf{p}_1 = \begin{bmatrix} -1 \\ -1 \end{bmatrix}, t_1 = 0 \right\} \left\{ \mathbf{p}_2 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, t_2 = 0 \right\} \left\{ \mathbf{p}_3 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}, t_3 = 1 \right\}.$$

The initial weight matrix and bias are

$$\mathbf{W}(0) = \begin{bmatrix} 1 & 0 \end{bmatrix}, b(0) = 0.5.$$

- i. Plot the initial decision boundary, weight vector and input patterns. Which patterns are correctly classified using the initial weight and bias?
- ii. Train the network with the perceptron rule. Present each input vector once, in the order shown.
- iii. Plot the final decision boundary, and demonstrate graphically which patterns are correctly classified.
- iv. Will the perceptron rule (given enough iterations) always learn to correctly classify the patterns in this training set, no matter what initial weights we use? Explain.

P4.3 We have a classification problem with four classes of input vector.
The four classes are

$$\text{class 1: } \left\{ \mathbf{p}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \mathbf{p}_2 = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \right\}, \text{ class 2: } \left\{ \mathbf{p}_3 = \begin{bmatrix} 2 \\ -1 \end{bmatrix}, \mathbf{p}_4 = \begin{bmatrix} 2 \\ 0 \end{bmatrix} \right\},$$

$$\text{class 3: } \left\{ \mathbf{p}_5 = \begin{bmatrix} -1 \\ 2 \end{bmatrix}, \mathbf{p}_6 = \begin{bmatrix} -2 \\ 1 \end{bmatrix} \right\}, \text{ class 4: } \left\{ \mathbf{p}_7 = \begin{bmatrix} -1 \\ -1 \end{bmatrix}, \mathbf{p}_8 = \begin{bmatrix} -2 \\ -2 \end{bmatrix} \right\}.$$

Design a perceptron network to solve this problem.

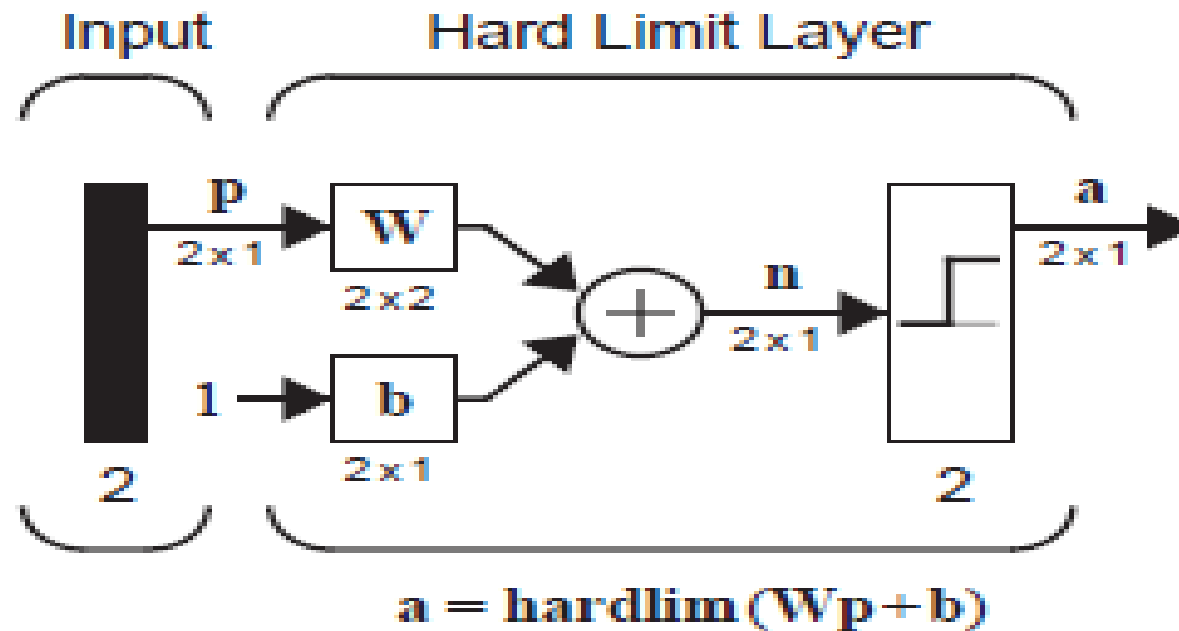
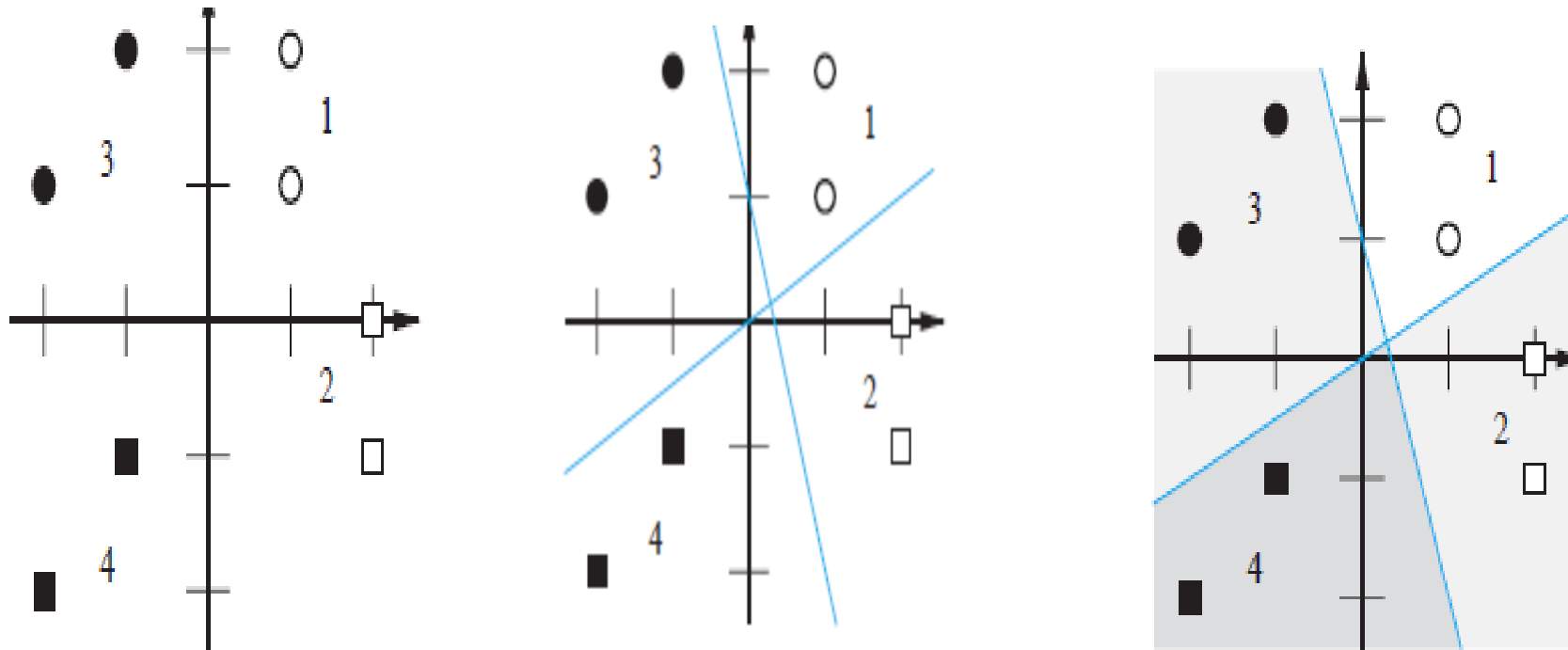


Figure P4.2 Two-Neuron Perceptron

To solve a problem with four classes of input vector we will need a perceptron with at least two neurons, since an S neuron perceptron can categorize 2^S classes.

A two-neuron perceptron creates two decision boundaries. Therefore, to divide the input space into the four categories



$${}_1\mathbf{w} = \begin{bmatrix} -3 \\ -1 \end{bmatrix} \text{ and } {}_2\mathbf{w} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}.$$

Note that the lengths of the weight vectors is not important, only their directions. They must be orthogonal to the decision boundaries. Now we can calculate the bias by picking a point on a boundary and satisfying Eq. (4.15):

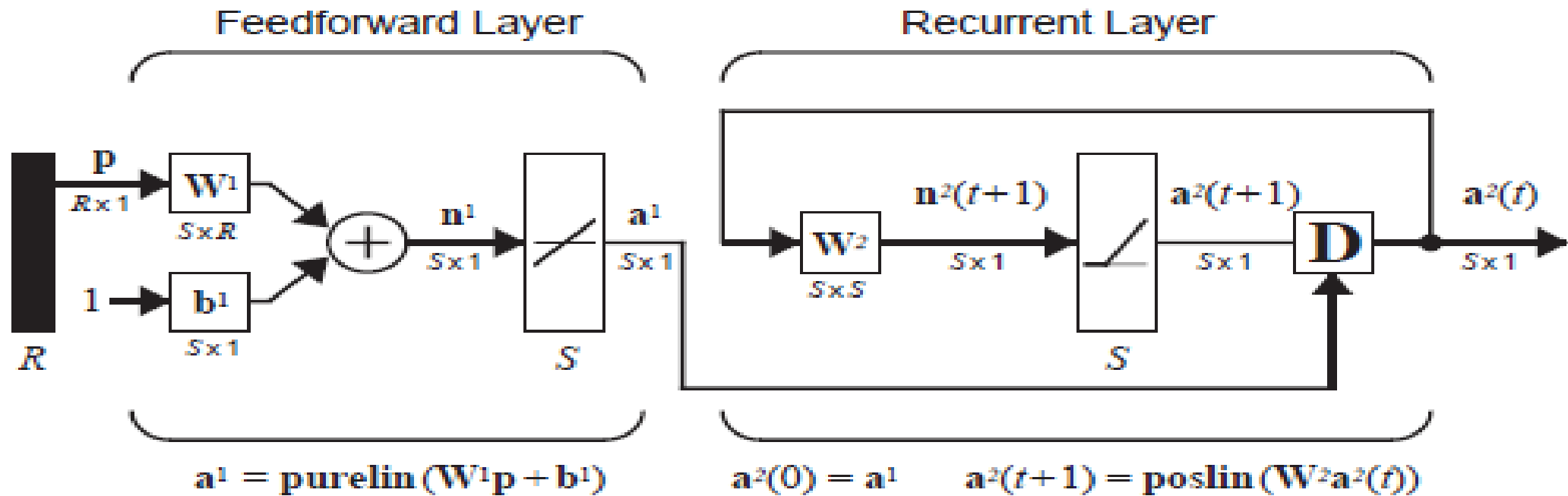
$$b_1 = -{}_1\mathbf{w}^T \mathbf{p} = -\begin{bmatrix} -3 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 1,$$

$$b_2 = -{}_2\mathbf{w}^T \mathbf{p} = -\begin{bmatrix} 1 & -2 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} = 0.$$

$$\mathbf{W} = \begin{bmatrix} {}_1\mathbf{w}^T \\ {}_2\mathbf{w}^T \end{bmatrix} = \begin{bmatrix} -3 & -1 \\ 1 & -2 \end{bmatrix} \text{ and } \mathbf{b} = \begin{bmatrix} 1 \\ 0 \end{bmatrix},$$

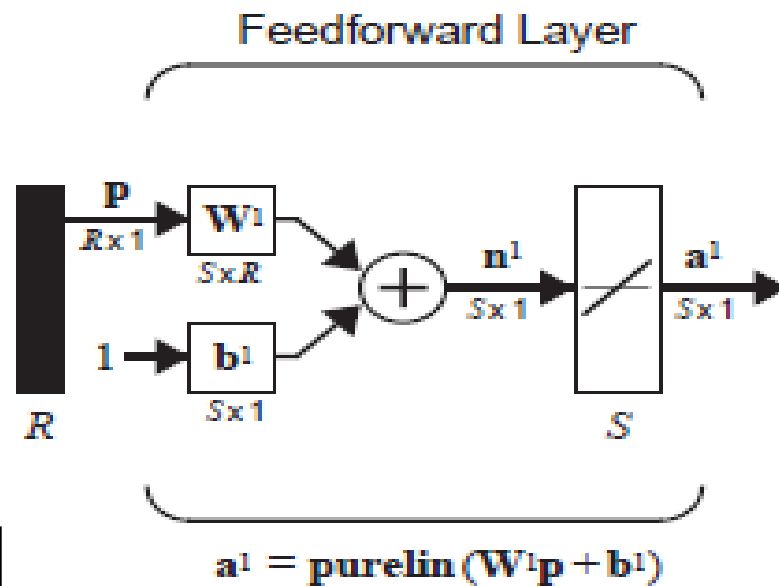
which completes our design.

Hamming Network



Feedforward Layer

For Banana/Apple Recognition



$$\mathbf{p}_1 = \begin{bmatrix} -1 \\ 1 \\ -1 \end{bmatrix}$$

$$\mathbf{p}_2 = \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix}$$

$$S = 2$$

$$\mathbf{W}^1 = \begin{bmatrix} \mathbf{p}_1^T \\ \mathbf{p}_2^T \end{bmatrix} = \begin{bmatrix} -1 & 1 & -1 \\ 1 & 1 & -1 \end{bmatrix}$$

$$\mathbf{b}^1 = \begin{bmatrix} R \\ R \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \end{bmatrix}$$

$$\mathbf{a}^1 = \mathbf{W}^1 \mathbf{p} + \mathbf{b}^1 = \begin{bmatrix} \mathbf{p}_1^T \\ \mathbf{p}_2^T \end{bmatrix} \mathbf{p} + \begin{bmatrix} 3 \\ 3 \end{bmatrix} = \begin{bmatrix} \mathbf{p}_1^T \mathbf{p} + 3 \\ \mathbf{p}_2^T \mathbf{p} + 3 \end{bmatrix}$$

First layer (Feedforward layer)

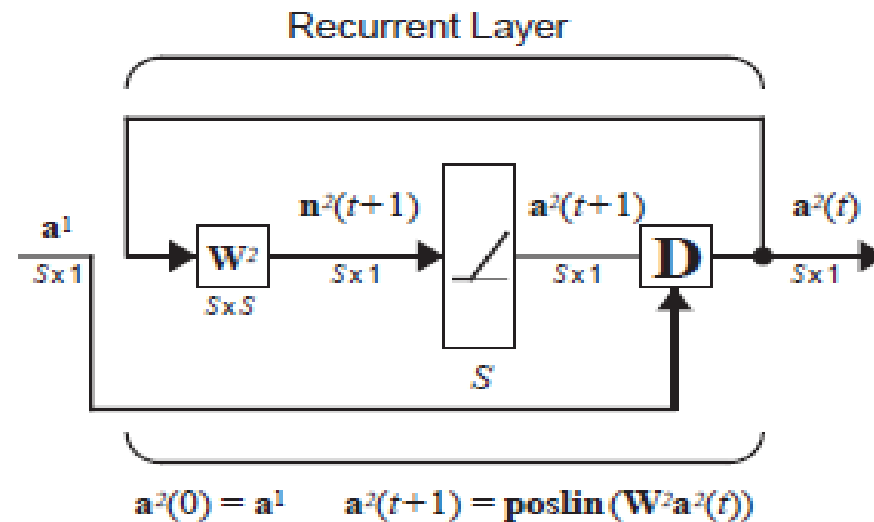
First Layer

Input (Rough Banana)

$$\mathbf{p} = \begin{bmatrix} -1 \\ -1 \\ -1 \end{bmatrix}$$

$$\mathbf{a}^1 = \begin{bmatrix} -1 & 1 & -1 \\ 1 & 1 & -1 \end{bmatrix} \begin{bmatrix} -1 \\ -1 \\ -1 \end{bmatrix} + \begin{bmatrix} 3 \\ 3 \end{bmatrix} = \begin{bmatrix} (1 + 3) \\ (-1 + 3) \end{bmatrix} = \begin{bmatrix} 4 \\ 2 \end{bmatrix}$$

Recurrent Layer



$$\mathbf{W}^2 = \begin{bmatrix} 1 & -\epsilon \\ -\epsilon & 1 \end{bmatrix} \quad \epsilon < \frac{1}{S-1}$$

$$\mathbf{a}^2(t+1) = \mathbf{poslin} \left(\begin{bmatrix} 1 & -\epsilon \\ -\epsilon & 1 \end{bmatrix} \mathbf{a}^2(t) \right) = \mathbf{poslin} \left(\begin{bmatrix} a_1^2(t) - \epsilon a_2^2(t) \\ a_2^2(t) - \epsilon a_1^2(t) \end{bmatrix} \right)$$

Second Layer

$$\mathbf{a}^2(1) = \mathbf{poslin}(\mathbf{W}^2 \mathbf{a}^2(0)) = \begin{cases} \mathbf{poslin}\left(\begin{bmatrix} 1 & -0.5 \\ -0.5 & 1 \end{bmatrix} \begin{bmatrix} 4 \\ 2 \end{bmatrix}\right) \\ \mathbf{poslin}\left(\begin{bmatrix} 3 \\ 0 \end{bmatrix}\right) = \begin{bmatrix} 3 \\ 0 \end{bmatrix} \end{cases}$$

$$\mathbf{a}^2(2) = \mathbf{poslin}(\mathbf{W}^2 \mathbf{a}^2(1)) = \begin{cases} \mathbf{poslin}\left(\begin{bmatrix} 1 & -0.5 \\ -0.5 & 1 \end{bmatrix} \begin{bmatrix} 3 \\ 0 \end{bmatrix}\right) \\ \mathbf{poslin}\left(\begin{bmatrix} 3 \\ -1.5 \end{bmatrix}\right) = \begin{bmatrix} 3 \\ 0 \end{bmatrix} \end{cases}$$

E3.1 In this chapter we have designed three different neural networks to distinguish between apples and oranges, based on three sensor measurements (shape, texture and weight). Suppose that we want to distinguish between bananas and pineapples:

$$\mathbf{p}_1 = \begin{bmatrix} -1 \\ 1 \\ -1 \end{bmatrix} \text{ (Banana)}$$

$$\mathbf{p}_2 = \begin{bmatrix} -1 \\ -1 \\ 1 \end{bmatrix} \text{ (Pineapple)}$$

- i. Design a perceptron to recognize these patterns.
- ii. Design a Hamming network to recognize these patterns.
- iii. Design a Hopfield network to recognize these patterns.
- iv. Test the operation of your networks by applying several different input patterns. Discuss the advantages and disadvantages of each network.